

# **ZAGAZIG UNIVERSITY**

## **FACULTY OF ENGINEERING**

### **ELECTRONICS AND COMMUNICATIONS**

#### **ENGINEERING**

#### **DEPARTMENT**



### **Information Security**

### **Caesar Cipher Report**

**Prepared by:**

- 1- Hisham Mohamed Abdelrahman**
- 2- Ali Mohamed Fawzy**
- 3- Adel Mohamed Ahmed**
- 4- Abdallah Mohamed Ahmed**
- 5- ziad Ahmed Abdelrahman**
- 6- ziad Ayman Abdulsalam**
- 7- Omar Talaat Mohamed Ahmed**
- 8- Ziad Ashmawy El-Demerdash**
- 9- Mohamed Ashraf Mohamed Embaby**

## Introduction:

This program implements the Caesar Cipher, which is one of the simplest classical encryption techniques. It works by shifting each letter in the text by a fixed number of positions (called *shift*) in the alphabet. The program can perform both encryption and decryption based on the user's choice

---

## *Python Implementation of the Caesar Cipher:*

### -Encryption

Concept: The code implements the Caesar Cipher, a simple encryption technique that shifts each alphabetic character by a given number (shift value). It converts each letter to its ASCII code, applies the shift for encryption or reverse shift for decryption, wraps within the alphabet using % 26, and then converts it back to a character. Non-alphabet characters remain unchanged. (  $C=(P+K)\text{mod}26$  )

The program asks the user for three inputs:

1. Text – the message that will be encrypted or decrypted.
2. Shift value – the number of positions each letter will be moved.

## Code Block

```
import sys

text = sys.argv[1]
key = int(sys.argv[2])

result = ""

for ch in text:
    if 'A' <= ch <= 'Z':
        result += chr((ord(ch) - 65 + key) % 26 + 65)
    elif 'a' <= ch <= 'z':
        result += chr((ord(ch) - 97 + key) % 26 + 97)
    else:
        result += ch

print(result)
```

---

### 1. Reading User Input

```
text = sys.argv[1]

key = int(sys.argv[2])

result = ""
```

- **text:** the message the user wants to encrypt.
- **key:** the number of positions each letter will be shifted.
- **result:** an empty string to store the final output.

**Purpose:** Read inputs from the command line and prepare a variable to store the encrypted text.

## 2. Looping Through Each Character

```
for ch in text:
```

- The program processes the text character by character.
  - It checks whether the character is:
    - an uppercase letter,
    - a lowercase letter,
    - or a non-alphabet character.
- 

## 3. Handling Uppercase Letters (A–Z)

```
if 'A' <= ch <= 'Z':
```

```
    result += chr((ord(ch) - 65 + key) % 26 + 65)
```

- Uppercase letters start from ASCII 65 ('A').
- `base = 65` ensures calculations stay within the A–Z range.

### Encryption Steps:

1. `ord(ch)`: converts the character into its ASCII value.
2. `- 65`: shifts the range to 0–25.
3. `+ key`: applies the encryption by shifting.
4. `% 26`: wraps around if the shift goes past 'Z'.
5. `chr(...)`: converts the final number back to a letter.

## 4. Handling Lowercase Letters (a–z)

```
elif 'a' <= ch <= 'z':
```

```
    result += chr((ord(ch) - 97 + key) % 26 + 97)
```

- Lowercase letters start at ASCII 97 ('a').
  - Same encryption logic as uppercase letters, but using the lowercase range.
- 

## 5. Non-Alphabet Characters

```
else:
```

```
    result += ch
```

Numbers, spaces, or symbols are added to the result unchanged.

Purpose: Only alphabetic letters are encrypted, other characters remain intact.

---

## 6. Printing the Final Output

```
print(result)
```

- Displays the final encrypted text after processing all characters.
- The output contains the shifted letters, while symbols and numbers remain the same.

## *Python Implementation of the Caesar Cipher:*

### **-Decryption**

**Concept:** Reverse the encryption process using:

$$(P=(C-K)\text{mod}26)$$

---

### **Code Block**

```
import sys

text = sys.argv[1]
key = int(sys.argv[2])

result = ""

for ch in text:
    if 'A' <= ch <= 'Z':
        result += chr((ord(ch) - 65 - key) % 26 + 65)
    elif 'a' <= ch <= 'z':
        result += chr((ord(ch) - 97 - key) % 26 + 97)
    else:
        result += ch

print(result)
```

## 1. Reading Input

```
text = sys.argv[1]

key = int(sys.argv[2])

result = ""
```

- text: the message to decrypt.
  - key: how many positions the letters will shift backward.
  - result: stores the final decrypted output.
- 

## 2. Loop Through Each Character

```
for ch in text:
```

- The program checks each character to see if it's a letter or not.
- 

## 3. Decrypting Letters (Uppercase & Lowercase Together)

```
if 'A' <= ch <= 'Z':

    result += chr((ord(ch) - 65 - key) % 26 + 65)

elif 'a' <= ch <= 'z':

    result += chr((ord(ch) - 97 - key) % 26 + 97)
```

Explanation (for both uppercase and lowercase):

- ord(ch) converts the letter to its ASCII code.
- Subtracting the base (65 for uppercase, 97 for lowercase) converts it to a 0–25 alphabet index.

- Subtracting the key shifts it backward (decryption).
  - % 26 keeps the result inside alphabet range.
  - Adding the base again returns it to its proper ASCII range.
  - chr() converts the number back to a decrypted character.
- 

## 4. Non-Letter Characters

```
else:
```

```
    result += ch
```

- Any character that isn't a letter (spaces, numbers, symbols) is kept as-is.
- 

## 5. Output

```
print(result)
```

- Prints the final decrypted string.
- 

## *Conclusion:*

The Caesar cipher is an excellent starting point for learning encryption.

Inter-grading Python code in the report demonstrates from theory to practical.



## ***Reference:***

- **Cryptography and Network Security Principles and Practice (William Stallings).**
- **mastering python for network.**